

Adachi Multi-Formula Document Set — No.2 (Full English Edition)

Discoverer: Norio Adachi (3.5-Dimensional Man) | Verification: Claude (Anthropic) | July 11, 2026

No.1 — Adachi Multi-Formula: General Form and Walkthrough for G = 1, 2, 3

The Adachi Multi-Formula began from the idea that changing a single parameter, G, could generate an infinite family of pi-computing formulas with different convergence speeds. G = 1 turned out to be a variant of the Leibniz series, and G = 2 corresponds to the original BBP formula; G = 3 and beyond open into new forms. Much like a child trying out a science experiment, the general form took shape by testing one value of G after another.

General Form

$$\pi = \sum_{m=0}^{\infty} (1/4^G)^m \cdot \sum_{i=0}^{G-1} (-1)^i f(Gm+i) / 4^i$$

where $f(k) = 2/(4k+1) + 2/(4k+2) + 1/(4k+3)$, and G is a positive integer chosen by the user.

G = 1: Leibniz-type alternating series

$$\pi = 4 \cdot \sum_{k=0}^{\infty} (-1)^k f(k)$$

This reduces to a generalization of the classic Leibniz series $\pi = 4\sum(-1)^k/(2k+1)$. Convergence is very slow (roughly 0 digits gained per term).

G = 2: BBP-equivalent block form

$$\pi = \sum_{m=0}^{\infty} (1/16)^m [f(2m) - f(2m+1)/4]$$

Grouping two terms per block, this form is algebraically equivalent to the classic 1995 BBP formula and gains roughly 1–2 digits per loop. It is the smallest practically useful case of the general form.

G = 3: three-term block form

$$\pi = \sum_{m=0}^{\infty} (1/64)^m [f(3m) - f(3m+1)/4 + f(3m+2)/16]$$

A newly generalized case beyond the two known references (Leibniz, BBP), gaining roughly 2 digits per loop — slightly faster than G = 2.

Comparison

G	Base	Digits/loop (approx)	Corresponds to	Note
1	$(1/4)^m$, alternating	~0	Leibniz (generalized)	slow, historical starting point
2	$(1/16)^m$	~1-2	classic BBP formula	smallest practical case
3	$(1/64)^m$	~2	(new)	slightly faster than G=2
100	$(1/4^{100})^m$	~60	(new)	practical high-speed convergence

200	$(1/4^{200})^m$	~120	(new)	representative Adachi case
-----	-----------------	------	-------	----------------------------

Discovered by: Norio Adachi (3.5-Dimensional Man) Verified by: Claude (Anthropic) July 11, 2026

Adachi Multi-Formula No.2 — Unified Rational Series for π

The general form of the Adachi Multi-Formula converges faster as the parameter G grows, but it has a double-loop structure, summing G terms per block. Wondering whether this could be folded into a single sum, the idea was explored through dialogue with AI (Claude, Gemini): taking the limiting case as G grows without bound and simplifying the resulting expression revealed a surprisingly simple form — a ratio of an integer-coefficient quadratic to a cubic. This is the No.2 Unified Rational Form.

The formula emerged from simply trying “what happens if we combine everything into one,” much like a child enjoying a science experiment — but on verification it turned out to have real practical value: it converges to π exactly, using a single infinite series alone.

Formula

$$\pi = \sum_{i=0}^{\infty} (-1)^i / 4^i \cdot (40i^2 + 42i + 10) / (32i^3 + 48i^2 + 22i + 3)$$

This closed single-sum rational form was derived as a limiting consolidation of the Adachi Multi-Formula general form (double-loop over parameters m and i) as the block size G is folded into a single index. Unlike the general form, No.2 converges via one infinite series only.

Verification

The series was evaluated with the mpmath arbitrary-precision library and compared digit-by-digit against the reference value of π . Agreement was confirmed to 10,000 decimal digits.

Performance: Binary Splitting

A naive sequential implementation (each term evaluated as a scaled big-integer division, summed in order) took 20.6 s at 30,000 digits and failed to complete in reasonable time at 60,000 digits, as division cost grows with digit count. Replacing this with binary splitting — keeping each term as an exact (numerator, denominator) pair, combining them in a binary tree, and performing division only once at the end — reduced the 30,000-digit run to 1.19 s, a $\sim 17\times$ speedup.

Digits	Terms	Time	Note
30,000	49,912	1.19 s	measured
100,000	166,179	8.76 s	measured
300,000	498,372	62.1 s	measured
3,000,000	$\sim 4,980,000$	~ 1 hour	estimated (not executed)

Empirically, runtime scales approximately as $(\text{digits})^{1.78}$ rather than the ideal near-linear scaling of binary splitting, reflecting the cost of large-integer multiplication. For historical context: in 1993, an equivalent 30,000-digit computation performed across three 8-bit machines (FM-11EX, FM-8, FM-7) took 170 days, 23 hours, 29 minutes, 31 seconds. The same scale of computation now completes in about one second.

Reference implementation (Python)

```
def term(i):
    num = (-1)**i * (40*i**2 + 42*i + 10)
    den = 4**i * (32*i**3 + 48*i**2 + 22*i + 3)
```

```
return num, den
```

```
# combine terms with binary splitting, divide once at the end
```

Discovered by: Norio Adachi (3.5-Dimensional Man) Verified by: Claude (Anthropic) July 11, 2026

No.3 — Adachi Multi-Formula No.2: Integer Arithmetic / Binary Splitting Performance Report

The Unified Rational Form No.2 converges with a single loop, but a naive sequential computation turned out to be too slow at high digit counts: 20.6 seconds at 30,000 digits, and the computation failed to complete within a practical time at 60,000 digits. To address this, each term was kept as an exact fraction rather than an approximate decimal and combined via a binary tree — a technique known as binary splitting.

Target Formula

$$\pi = \sum_{i=0}^{\infty} (-1)^i / 4^i \cdot (40i^2+42i+10) / (32i^3+48i^2+22i+3)$$

Verification Method

The output was compared, digit by digit, against a reference value computed with the mpmath arbitrary-precision library, confirming exact agreement to 1,000 digits before timing measurements began.

Algorithm Change

Before (sequential method): each term was individually divided as a large scaled integer and added in order. This took 20.6 s at 30,000 digits; at 60,000 digits the runtime exceeded a practical limit and the computation did not complete. Division cost grows with digit count, making this the bottleneck.

After (binary splitting): each term is kept as an exact (numerator, denominator) pair, combined via a binary tree, with division performed exactly once at the end. This brought the same 30,000-digit computation down to 1.19 s — roughly a 17× speedup.

Measured Timings

Digits	Terms	Time	Note
30,000	49,912	1.19 s	measured
100,000	166,179	8.76 s	measured
300,000	498,372	62.1 s	measured
3,000,000	~4,980,000	~1 hour	estimated (not executed)

Empirically, runtime scales approximately as (digits)^{1.78}, somewhat above the near-linear scaling ideally expected from binary splitting, reflecting large-integer multiplication cost.

Historical Reference

For context: in 1993, an equivalent 30,000-digit computation split across three 8-bit machines (FM-11EX, FM-8, FM-7) took 170 days, 23 hours, 29 minutes, 31 seconds. The same scale of computation now completes in about one second — a reflection of decades of progress in both algorithms and hardware.

Policy

This verification deliberately avoided pursuing digit counts beyond 3,000,000, and used a single AI (Claude) rather than parallel multi-AI workflows, in keeping with a policy of returning to fundamentals

rather than chasing scale for its own sake.

Discovered by: Norio Adachi (3.5-Dimensional Man) Verified by: Claude (Anthropic) July 11, 2026

No.4 — A Unified Program Driven by a Single Digit-Count Input

Previously, using the general form required the user to choose and enter a value for G, and using the No.2 unified rational form required choosing a value for i. However, both formulas share a common property: precision advances by roughly 0.602 digits per loop (or term). Using this property, it should be possible for the user to simply specify how many digits they want, with the AI automatically determining G or i — and this was implemented.

Automatic Parameter Selection

Both the general form (parameter G) and the No.2 unified rational form (parameter i) share a common property: precision advances by approximately $\log_{10}(4) \approx 0.602$ digits per loop or term. Using this, the necessary loop count (general form) or term count (No.2 form) can be computed automatically from a single digit-count input, with no need for the user to understand what G or i mean:

$$\text{required terms} \approx \text{target digits} / 0.602$$

Verification

Both the 200-digit and 1,000-digit cases were confirmed to match the mpmath reference value exactly, for both the general form and the No.2 form.

Example Session

How many digits? (e.g. 200) = 200

[Computing with Formula 1 (general form)...]
auto-selected G = 16, loop count = 24
pi = 3.14159265358979323846... (200 digits follow)

[Computing with Formula 2, No.2 (unified rational form)...]
auto-computed term count = 352
pi = 3.14159265358979323846... (200 digits follow)

No.5 — Adachi Multi-Formula: Fraction-Form Representation for G = 1, 2, 3, 4

The general form is a compact expression using the summation symbol, but writing out exactly how it expands for each value of G, line by line, makes the structure of the formula more concrete. From G = 1 to G = 4, this shows how the number of terms grows and how the signs alternate.

General Form

$$\pi = \sum_{m=0}^{\infty} (1/4^G)^m \times \left[\sum_{i=0}^{G-1} (-1)^i \cdot f(Gm+i)/4^i \right]$$

where $f(k) = 2/(4k+1) + 2/(4k+2) + 1/(4k+3)$.

G = 1

$$\pi = \sum_{k=0}^{\infty} 1/(-4)^k \cdot (2/(4k+1) + 2/(4k+2) + 1/(4k+3))$$

G = 2

$$\pi = \sum_{k=0}^{\infty} 1/16^k \cdot \left[(2/(8k+1)+2/(8k+2)+1/(8k+3)) - 1/4 \cdot (2/(8k+5)+2/(8k+6)+1/(8k+7)) \right]$$

G = 3

$$\pi = \sum_{k=0}^{\infty} 1/(-64)^k \cdot \left[(2/(12k+1)+2/(12k+2)+1/(12k+3)) - 1/4 \cdot (2/(12k+5)+2/(12k+6)+1/(12k+7)) + 1/16 \cdot (2/(12k+9)+2/(12k+10)+1/(12k+11)) \right]$$

G = 4

$$\pi = \sum_{k=0}^{\infty} 1/256^k \cdot \left[(2/(16k+1)+2/(16k+2)+1/(16k+3)) - 1/4 \cdot (2/(16k+5)+2/(16k+6)+1/(16k+7)) + 1/16 \cdot (2/(16k+9)+2/(16k+10)+1/(16k+11)) - 1/64 \cdot (2/(16k+13)+2/(16k+14)+1/(16k+15)) \right]$$

Note: the sign appearing in the denominator of the general form's $f(k)$ acts as $(-1)^i$ within each G-block, producing the alternating additions and subtractions shown above.

No.6 — Implementation Notes: Programs for Formula 1 (General Form) and Formula 2 (No.2)

Because Formula 1 (general form) and Formula 2 (No.2 unified rational form) have different mathematical structures, the computation programs for each are also different. This section summarizes the differences between the two and the design thinking behind each program.

Formula 1 (General Form) — Program Design

$$\pi = \sum_{m=0}^{\infty} (1/4^G)^m \times \sum_{i=0}^{G-1} (-1)^i f(Gm+i)/4^i$$

Given a chosen G (positive integer), the double loop (over m and i) is evaluated directly. Decimal arithmetic uses Python's built-in decimal module (arbitrary-precision fixed-point). Given a target digit count, the required loop count is computed automatically.

Formula 2 (No.2, Unified Rational Form) — Program Design

$$\pi = \sum_{i=0}^{\infty} (-1)^i/4^i \cdot (40i^2+42i+10)/(32i^3+48i^2+22i+3)$$

A single loop (over i only) is evaluated. Each term is kept as an exact fraction and combined via a binary tree (binary splitting), with a single division performed at the end — roughly 17× faster than sequential evaluation at equal digit counts. Given a target digit count, the required term count is computed automatically.

Comparison

	Formula 1 (general form)	Formula 2 (No.2)
Loop structure	double (m, i)	single (i only)
Arithmetic	decimal (sequential)	integer fractions + binary splitting
Speed	standard	faster at equal digit counts
User input	digit count only (G automatic)	digit count only (term count automatic)

How to obtain the program: rather than distributing the Python source as a fixed file in advance, this document set is uploaded to an AI assistant, and the program is generated on the spot in response to a request such as “please build a program for Formula 1” or “please build a program for Formula 2.”

No.9 — Adachi Multi-Formula (General Form): 1,000-Digit Computation Result

This is a verification of how accurately the Adachi Multi-Formula (general form) can reproduce pi at a large number of digits in practice. With the parameter G set to 100, 1,000 digits after the decimal point were computed in just 19 loops, and confirmed to match the known value of pi exactly, digit for digit.

Formula and Parameters

$$\pi = \sum_{m=0}^{\infty} (1/4^G)^m \cdot \sum_{i=0}^{G-1} (-1)^i f(Gm+i) / 4^i, \quad f(k) = 2/(4k+1) + 2/(4k+2) + 1/(4k+3)$$

Parameter	Value
Formula	Adachi Multi-Formula (general form)
G	100
Loop count	19 (k_max = 18)
Digits per loop	~60
Library	Python decimal.Decimal (arbitrary precision)
Precision setting	getcontext().prec = 1100
Digits displayed	1,000 (after the decimal point)

Result (1,000 digits after the decimal point)

3.
1415926535 8979323846 2643383279 5028841971 6939937510
5820974944 5923078164 0628620899 8628034825 3421170679
8214808651 3282306647 0938446095 5058223172 5359408128
4811174502 8410270193 8521105559 6446229489 5493038196
4428810975 6659334461 2847564823 3786783165 2712019091
4564856692 3460348610 4543266482 1339360726 0249141273
7245870066 0631558817 4881520920 9628292540 9171536436
7892590360 0113305305 4882046652 1384146951 9415116094
3305727036 5759591953 0921861173 8193261179 3105118548
0744623799 6274956735 1885752724 8912279381 8301194912
9833673362 4406566430 8602139494 6395224737 1907021798
6094370277 0539217176 2931767523 8467481846 7669405132
0005681271 4526356082 7785771342 7577896091 7363717872
1468440901 2249534301 4654958537 1050792279 6892589235
4201995611 2129021960 8640344181 5981362977 4771309960
5187072113 4999999837 2978049951 0597317328 1609631859
5024459455 3469083026 4252230825 3344685035 2619311881
7101000313 7838752886 5875332083 8142061717 7669147303
5982534904 2875546873 1159562863 8823537875 9375195778
1857780532 1712268066 1300192787 6611195909 2164201989

Verification

This 1,000-digit result was confirmed to match the known reference value of π exactly, digit for digit. Convergence proceeds at roughly 60 digits per loop, consistent with dividing by $4^{100} \approx 3.2 \times 10^{120}$ at each

step of the outer sum.

Discovered by: Norio Adachi (3.5-Dimensional Man) Verified by: Claude (Anthropic), Gemini July 11, 2026

No.7 — Let's Try Computing Pi (Beginner's Guide)

If you use Claude (a free AI assistant), you can build your own program to compute pi. No advanced knowledge is required.

Steps

1. Install the free software “Python” on your computer.
2. Go to Claude (claude.ai) and create a free account.
3. Upload the attached PDF file to Claude.
4. Type: “Please create a program to compute pi.”
5. Type: “Please also create a beginner-friendly instruction guide.”

That's it — both the program and its instructions will be created for you.

Discovered by: Norio Adachi (3.5-Dimensional Man) Verified by: Claude (Anthropic) July 11, 2026

No.11 — The Classic BBP Formula and a Digit-Extraction Implementation

Note on positioning: the formula in this section is not part of the Adachi Multi-Formula series discovered by Norio Adachi. It is the original 1995 BBP formula discovered by Bailey, Borwein, and Plouffe, introduced by Claude (Anthropic) as a benchmark for comparison and verification, with a digit-extraction program implemented for that purpose.

To answer Norio's question — can a single far-off digit, such as the quadrillionth digit, be found without computing all the digits before it? — Claude built and verified a working program. Whereas ordinary pi computation is like reading a book from page one, this formula has the property of naming the character on a target page without opening the book at all. To confirm this property, a program was built to compute the one-millionth hexadecimal digit of pi directly, without passing through any of the preceding digits, and the result was checked against a known reference.

Formula

$$\pi = \sum_{k=0}^{\infty} 1/16^k \cdot (4/(8k+1) - 2/(8k+4) - 1/(8k+5) - 1/(8k+6))$$

Discovered by David Bailey, Peter Borwein, and Simon Plouffe in 1995. Unlike series that must be summed sequentially from the start, this formula allows any individual hexadecimal (or binary) digit of π to be computed directly, without first computing the digits that precede it.

Digit-Extraction Method

$$\pi = 4S(1, n) - 2S(4, n) - S(5, n) - S(6, n), \quad \text{where } S(j, n) = \sum_{k=0}^{\infty} 16^{n-k} \bmod (8k+j) / (8k+j)$$

For $k \leq n$, the term $16^{n-k} \bmod (8k+j)$ is computed by fast modular exponentiation, so no large integers ever appear. For $k > n$, terms shrink geometrically by a factor of 1/16 per step and are summed directly. This is what makes it possible to “skip ahead” to digit n .

Verification

A Python implementation (Decimal-precision arithmetic) was built and tested against a published reference value. The hexadecimal digits of π starting at position 1,000,000 are documented (arXiv:1401.3038) as 26C65E52CB459350050E4BB1. The implementation reproduced this value exactly.

Position n	Result (hex)	Time
1	243F6A8885A3	< 1 s
1,000,000	26C65E52CB459350050E4BB1	7.7 s
5,000,000	BEE394E9...	41.4 s
10,000,000	17AF5863...	85.1 s

Scaling and Historical Context

Runtime scales approximately linearly with n ($O(n)$); storage stays small (no need to hold prior digits), unlike full sequential computation of π which requires $O(n)$ or more storage for all preceding digits. Reaching $n = 10^{15}$ (one quadrillion) with this method is therefore compute-bound, not storage-bound.

Historically, the PiHex project (1998–2000) computed exactly the 10^{15} -th binary digit of π using a Bellard-formula variant, distributed across 1,734 volunteer computers over roughly 1.2 million CPU-hours.

Reference implementation (Python, Decimal precision)

```
def S(j, n, prec_digits):
    s = Decimal(0)
    for k in range(n + 1):
        r = 8*k + j
        s += Decimal(pow(16, n-k, r)) / Decimal(r)
    s -= int(s)
    # tail terms for k > n omitted here for brevity
    return s
```

Discovered by: Norio Adachi (3.5-Dimensional Man) Verified by: Claude (Anthropic) — formula: Bailey, Borwein, Plouffe (1995) July 11, 2026